

Packaging a Machine Learning Prediction Service with Docker

Iris species prediction API (Flask + scikit-learn), images iris-predict:v1 and iris-predict:v2

Threshala, BSc in Applied Data Science Communication, KDU | Group members: D/ADC/25/0006, D/ADC/25/0022, D/ADC/25/0043 | 29 September 2026

1. Overview

This report documents how a small machine learning application was packaged into a Docker image so that anyone can run it and request a prediction. The application trains a decision tree on the classic Iris dataset, saves the trained model to disk, and serves predictions through a Flask HTTP API. Two image versions were built from the same Dockerfile: v1 (original) and v2 (changed home message). Both were run as the container iris-lab and tested with curl.

2. How the application works

The three provided files form one pipeline. The route from input JSON to prediction is shown in Figure 1 and summarised in the checkpoint table below.

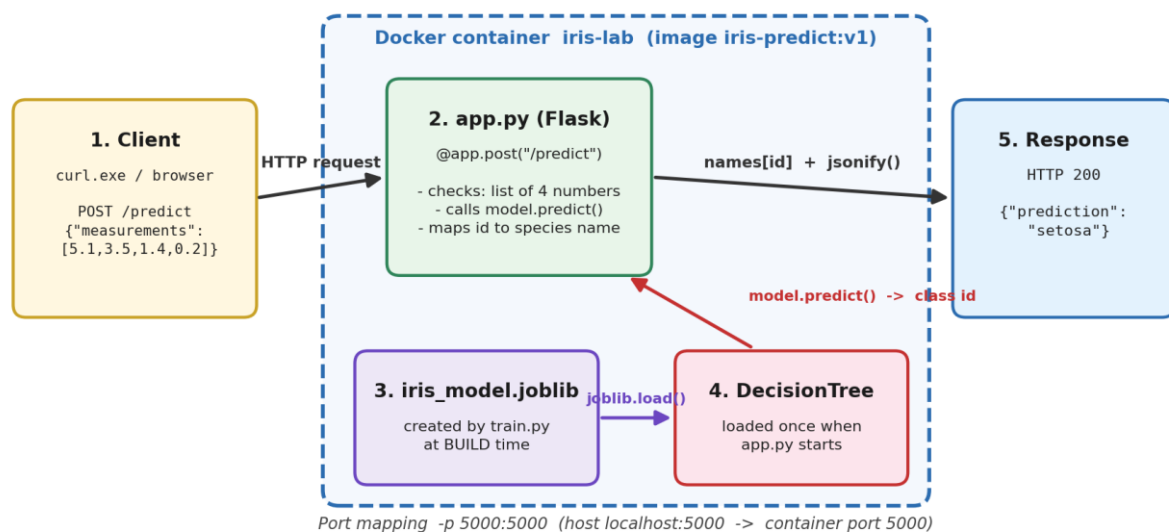


Figure 1. Request flow: input JSON, Flask app, saved model and prediction response.

Checkpoint	Where	What happens
Saved model	train.py	Trains DecisionTreeClassifier(max_depth=3, random_state=42) on load_iris() and saves it with joblib.dump() as iris_model.joblib, next to the script.
HTTP endpoint	app.py	On startup, joblib.load() reads the model. GET / describes the service. POST /predict accepts JSON of the form {"measurements": [4 numbers]}.

Checkpoint	Where	What happens
Response	app.py	Valid input: the predicted class id (0, 1 or 2) is mapped to setosa, versicolor or virginica and returned as {"prediction": ...} with HTTP 200. Invalid input: {"error": "measurements must be four numbers"} with HTTP 400.
Dependencies	requirements.txt	Flask (3.1.x), scikit-learn (1.7.x) and joblib (1.5.x), each pinned to a compatible range.

Input validation. Before predicting, the app checks that the body is a JSON object, that measurements is a list of exactly four items, and that every item is an int or float. Booleans are explicitly rejected because Python treats True and False as integers. Anything else returns 400 instead of crashing the server.

3. The Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY train.py .
COPY app.py .

RUN python train.py

EXPOSE 5000

CMD ["python", "app.py"]
```

Instruction	Explanation
FROM python:3.12-slim	Starts from the official Python image. The slim variant leaves out extra system packages, keeping the base smaller.
WORKDIR /app	Sets /app as the working directory for all later instructions and for the running container.
COPY requirements.txt .	Copies only the dependency list first. Docker caches each layer, so the slow pip install layer is reused as long as this file is unchanged, even when app.py is edited.
RUN pip install --no-cache-dir ...	Build step. Installs Flask, scikit-learn and joblib into the image. --no-cache-dir avoids storing pip's download cache in the layer.
COPY train.py . / COPY app.py .	Copies the source code after the dependencies, so code edits only invalidate layers from this point on.
RUN python train.py	Build step. Trains the model and writes iris_model.joblib into /app. The file becomes part of the image, so it exists before any container starts.

Instruction	Explanation
EXPOSE 5000	Documents that the app listens on port 5000. It does not publish the port; that is done by -p at run time.
CMD ["python", "app.py"]	Run-time command. Executed only when a container starts, launching the Flask server on 0.0.0.0:5000.

Build time versus run time

Build time (docker build): FROM, COPY and RUN instructions execute once, and each produces a read-only image layer. This is when packages are installed and train.py creates the model. **Run time** (docker run): Docker creates a container from the finished image and executes the single CMD. app.py loads the model that is already in the image and starts serving. Training at build time means every container from the same image uses an identical model and starts quickly, with no training delay.

Why host 0.0.0.0? app.py calls app.run(host="0.0.0.0", port=5000). Inside a container, the default 127.0.0.1 would only accept connections from within the container itself, so the port mapping from the host could never reach it.

4. Build, run and verify

```
docker build -t iris-predict:v1 .
docker run -d --name iris-lab -p 5000:5000 iris-predict:v1
docker ps
docker logs iris-lab
curl.exe http://localhost:5000/
```

Evidence for this section comes from two machines. The first (Figures 2, 8, 9 and 10) was used for the v1 and v2 tests. The second (Figures 3 to 7) shows the build output, the image listing, the running containers and the GET requests. Image IDs differ between the two machines because each built its own image from the same files.

GET / on the first machine. The v1 container returned the service description in the browser (Figure 2).

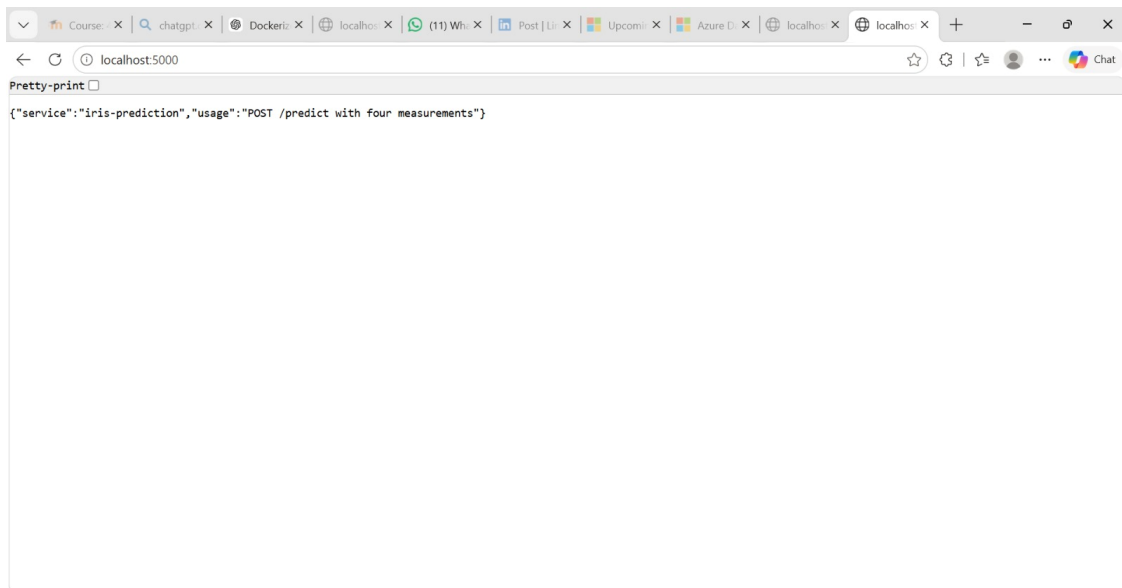


Figure 2. GET / on the v1 container: {"service":"iris-prediction","usage":"POST /predict with four measurements"}.

4.1 Build

The build finished successfully and Docker printed a link to the build details in Docker Desktop.

```
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/lyz4g09ldvdfw5nfnznpqteyok
PS C:\Users\HP\Desktop\AML\Practical\Week 11\ML prediction application in Docker>
```

Figure 3. End of a successful docker build for iris-predict:v1.

4.2 Image listing

docker images shows iris-predict:v1 with two sizes: 559 MB of disk usage and 130 MB of content size. Disk usage is the space the image occupies locally once unpacked, while content size is the compressed size of its layers. This is why other tools quote about 130 MB and Docker Desktop quotes about 559 MB for the same image.

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
beanstalk-demo:latest	93ed87412278	196MB	47.3MB	U
gcr.io/k8s-minikube/kicbase@sha256:4a1c825b61479e6c898851ea66f13c620aaeab6002746e95067fc2c4b38a0b24	4a1c825b6147	1.89GB	532MB	U
iris-predict:v1	976383dfd1b0	559MB	130MB	U
kicbase/stable:v0.0.51	4a1c825b6147	1.89GB	532MB	U

PS C:\Users\HP\Desktop\AML\Practical\Week 11\ML prediction application in Docker>

Figure 4. docker images: iris-predict:v1, 559 MB disk usage, 130 MB content size.

4.3 Running containers

docker ps shows the new container iris-lab created from iris-predict:v1 with the mapping 0.0.0.0:5000->5000/tcp, meaning host port 5000 is forwarded to container port 5000. The COMMAND column shows python app.py, which is the CMD from the Dockerfile. A second container, mystifying_mcnulty, also from iris-predict:v1, had been running for 7 minutes. Its name

was generated by Docker, so it was started without --name, and it shows only 5000/tcp with no host mapping, so it is not reachable from the host.

```
>> docker ps
iris-lab
9d12ab020943d38ce27fa1c32cc0756a996ee560fe7f1317186b14711835d1df
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
TS            NAMES
9d12ab020943   iris-predict:v1 "python app.py"         Less than a second ago    Up Less than a second    0.0
.0.0:5000->5000/tcp, [::]:5000->5000/tcp   iris-lab
26bca9b4ae16   iris-predict:v1 "python app.py"         7 minutes ago            Up 7 minutes            500
0/tcp                                     mystifying_mcnulty
```

Figure 5. docker ps: iris-lab (port 5000 published) and an earlier unnamed container (port not published).

4.4 GET request

The service was checked from the browser and from PowerShell. Both return the same JSON. In PowerShell, Invoke-WebRequest reports StatusCode 200, Content-Type application/json and Content-Length 77, which is the length of the JSON body. The Server header shows the app is running Flask's Werkzeug server on Python 3.12.

```
Pretty-print ☐
{"service":"iris-prediction","usage":"POST /predict with four measurements"}
```

Figure 6. GET / in the browser on the second machine.

```
StatusCode      : 200
StatusDescription : OK
Content         : {"service":"iris-prediction","usage":"POST /predict with four measurements"}

RawContent      : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 77
                  Content-Type: application/json
                  Date: Tue, 29 Sep 2026 06:55:52 GMT
                  Server: Werkzeug/3.1.9 Python/3.12.14
```

Figure 7. GET / from PowerShell: HTTP 200 with the service description as JSON.

5. Prediction requests

5.1 Valid request

Windows PowerShell was used, so curl.exe was called (plain curl is an alias for Invoke-WebRequest) and the JSON quotes were escaped.

```
Terminal
PS C:\Users\ASUS\OneDrive\Desktop\iris-docker> curl.exe -i -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"me
asurements":[5.1,3.5,1.4,0.2]}'
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.12.14
Date: Tue, 29 Sep 2026 05:37:30 GMT
Content-Type: application/json
Content-Length: 24
Connection: close

{"prediction":"setosa"}
PS C:\Users\ASUS\OneDrive\Desktop\iris-docker>
```

Figure 8. Valid POST /predict with the measurements 5.1, 3.5, 1.4 and 0.2.

Item	Result
Status	HTTP/1.1 200 OK
Headers	Content-Type: application/json, Content-Length: 24, Server: Werkzeug/3.1.9 Python/3.12.14
Body	<code>{"prediction": "setosa"}</code>
Meaning	200 means the request was understood and processed. The four values (sepal length 5.1, sepal width 3.5, petal length 1.4, petal width 0.2) are typical of Iris setosa, which the tree classifies as class 0.

5.2 Invalid request

```
curl.exe -i -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"measurements\":[5.1,3.5]}'
```

Only two measurements are sent, so the length check in app.py fails before the model is ever called. From the code, the expected response is:

Item	Result
Status	HTTP/1.1 400 BAD REQUEST
Body	<code>{"error": "measurements must be four numbers"}</code>
Meaning	400 signals a client error: the request was well-formed HTTP but the data does not meet the API's rules. The server itself is fine. The fix is on the caller's side (send exactly four numbers under the key measurements).

Evidence still needed: paste your own terminal screenshot of this invalid request. The table above is derived from app.py, not from a captured run, and the submission asks for both terminal outputs.

6. Changing the app and rebuilding (v2)

The home message in app.py was changed so that GET / also returns a message field. The old container was stopped and removed, the image rebuilt with a new tag, and a new container started from it:

```
docker stop iris-lab
docker rm iris-lab
docker build -t iris-predict:v2 .
docker run -d --name iris-lab -p 5000:5000 iris-predict:v2
```

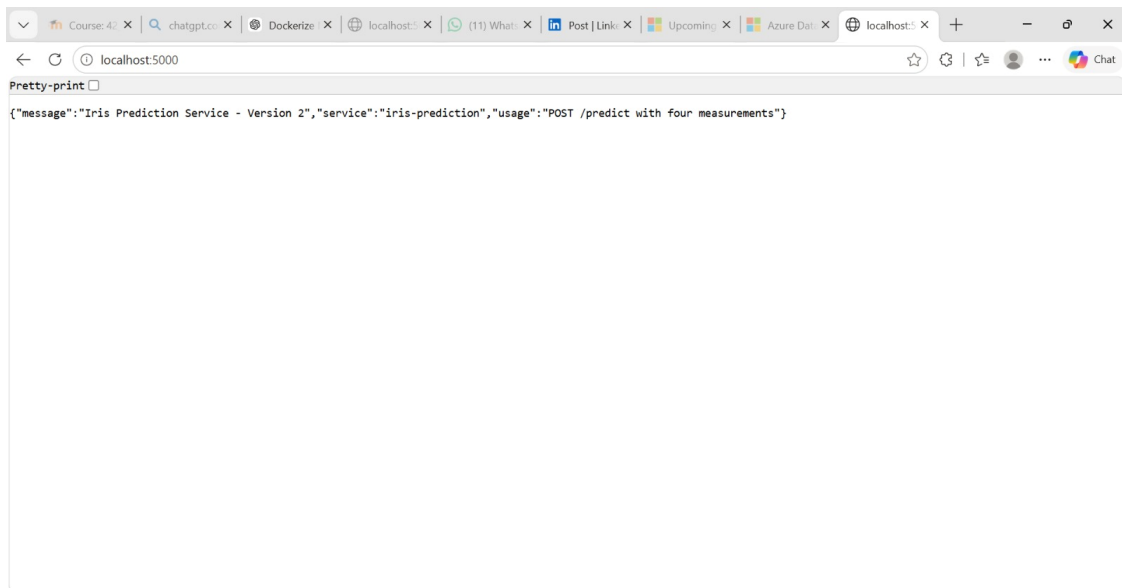


Figure 9. GET / on the v2 container now includes "message": "Iris Prediction Service - Version 2".

Docker Desktop lists both images side by side. They have different image IDs, showing that the v1 image was not modified by the rebuild:

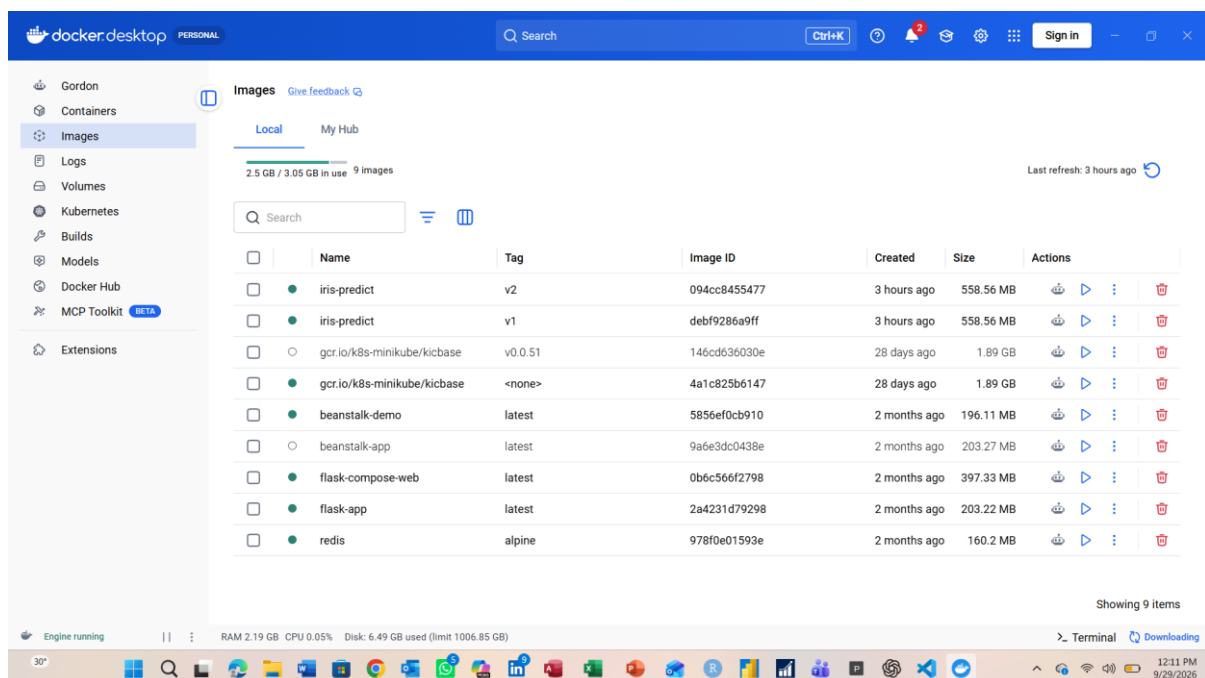


Figure 10. Docker Desktop, Images tab: iris-predict:v2 (094cc8455477) and iris-predict:v1 (deb9286a9ff), each 558.56 MB.

Why a rebuild is necessary. A running container is created from an image, and the image contains a frozen copy of app.py made by COPY at build time. Editing app.py on the host changes nothing inside the image or any container already running from it. Only docker build copies the new file into a new image layer, and a new container must then be started from that image. Because the COPY app.py layer changed, Docker rebuilt from that layer onward and reused the cached pip install layer, which is why v2 builds quickly.

7. What the image contains and what happens at start-up

The image iris-predict contains the Python 3.12 slim base, the installed packages (Flask, scikit-learn, joblib and their dependencies), the two source files train.py and app.py in /app, and the trained model file iris_model.joblib that train.py produced during the build. When a container starts, Docker adds a thin writable layer on top of the image and runs the CMD, python app.py. The app loads the model file from the image with joblib.load(), starts the Flask server on 0.0.0.0:5000, and waits for requests. The -p 5000:5000 option forwards traffic from port 5000 on the host to port 5000 in the container. A code change requires a rebuild because the source is baked into the image at build time. Containers do not read the host's files, so the new code only reaches a container through a new image.

8. Problem diagnosed

Figure 5 shows two containers from the same image running at once: iris-lab, and an earlier container with the auto-generated name mystifying_mcnulty. The earlier one was started without --name and without -p, so Docker gave it a random name and left port 5000 unpublished.

Part	Finding
Symptom	The service could not be reached on localhost:5000 from that first container, and docker ps showed a container with a random name and no host port mapping (5000/tcp only).
How found	docker ps: the PORTS column had no 0.0.0.0:5000->5000 entry for the first container, and its NAMES value was not iris-lab.
Fix	Start the container with both a name and a port mapping: docker run -d --name iris-lab -p 5000:5000 iris-predict:v1. The extra container can be removed with docker stop mystifying_mcnulty followed by docker rm mystifying_mcnulty.

Please confirm: I inferred this from the docker ps screenshot. If your group's real problem (or the one you helped another group with) was different, replace this table. Include the command you used, such as docker logs iris-lab or docker ps -a.

9. Observations

- **Image size.** docker images shows 559 MB disk usage and 130 MB content size for iris-predict:v1 (Docker Desktop shows 558.56 MB). The image is large mainly because scikit-learn brings in NumPy and SciPy. A smaller build would need a different base or fewer dependencies.
- **Development server.** app.py uses Flask's built-in server, which is fine for the lab but not for production. A WSGI server such as gunicorn would be the next step.
- **Model fixed at build time.** Retraining requires rebuilding the image, which guarantees reproducibility but not freshness.
- **Reproducibility.** Pinned version ranges in requirements.txt and a small, ordered Dockerfile mean another group can rebuild the same service from the four project files.

10. Cleanup

```
docker stop iris-lab  
docker rm iris-lab  
docker stop mystifying_mcnulty  
docker rm mystifying_mcnulty
```

The last two commands remove the earlier unnamed container. If a v2 container was created, stop and remove that one too. Images are kept until the evidence has been marked.